Explicação da Onda 0 — AutoLoad V4

> **Para quem é este documento:** alguém que está começando na área de tecnologia e quer entender, sem jargão, o que é a "Onda 0" do plano de observabilidade do AutoLoad V4, o que cada etapa faz e em que pé ela está.

Data	17/06/2026
Produto	AutoLoad V4 (backend .NET 8 · frontend Next.js 14 · EdgeGateway .NET 8 Worker · portal-cliente)
Ferramenta de monitoramento	Sentry (org `automind-qd`)
Fonte de verdade do status	Auditoria de código (08/06/2026) — confronto etapa-a-etapa contra o código real em `Produtos/Autoload V4`
Fontes lidas	Cronograma Revisado (Excel 02/06), Auditoria de código (08/06), Documentação do produto, Ata de alinhamento e Questionário do dev

O que é a Onda 0?

Imagine que você vai construir uma casa de vários andares. Antes de levantar qualquer parede, você precisa de uma **fundação firme e de um terreno limpo**. A Onda 0 é exatamente isso para o plano de observabilidade do AutoLoad V4: é a **fase de saneamento da base**.

"Observabilidade" é a capacidade de **enxergar o que está acontecendo dentro do sistema** — saber quando algo quebra, onde quebrou e por quê. A ferramenta que faz isso aqui é o **Sentry**, que funciona como uma "central de alarmes" que captura os erros do programa.

O problema é que, hoje, essa central está com os alarmes **mal regulados**: dispara milhares de vezes para uma única falha (chega a projetar centenas de milhares de eventos por mês de **um só** erro), mistura dados de quatro frentes diferentes no mesmo lugar e pode estar deixando passar informações pessoais — CPF, placa, dados de motorista — que não deveriam aparecer. A Onda 0 conserta tudo isso **antes** de seguir em frente.

Por isso ela é chamada de **"gate"** (portão): enquanto a Onda 0 não estiver concluída, as próximas ondas (1 a 6) **não devem começar**. Não adianta construir andares novos sobre uma fundação torta.

O AutoLoad V4 tem **quatro "motores"**: o **backend** (.NET 8, o cérebro), o **frontend** (Next.js, a tela que o operador vê), o **EdgeGateway** (um programa .NET que roda perto dos equipamentos de campo e fala com eles via MQTT) e o **portal-cliente** (a área do cliente). A Onda 0 tem **8 etapas**. Abaixo, cada uma é explicada em linguagem do dia a dia.

> **Uma diferença importante em relação ao AutoChecker:** o AutoLoad **não tem um quadro de tarefas no Azure DevOps** para servir de fonte de verdade. Por isso, aqui o status de cada etapa vem da **auditoria de código** feita em 08/06/2026, que abriu o repositório e confrontou, arquivo por arquivo, o que o cronograma afirma contra o que o código realmente faz. Onde a verdade depende do painel do Sentry ou do que acontece em produção (e não do código), isso está marcado como **"não verificável no código"** — não é um erro, é apenas algo que o repositório não consegue provar.

Etapa 1 — Padronizar a etiqueta `environment` nos 4 SDKs

O que é essa etapa?

Todo erro que chega ao Sentry vem com uma **etiqueta** dizendo de onde ele veio: do ambiente de **produção** (onde estão os clientes reais), de **homologação** (testes) ou da **máquina do próprio desenvolvedor**. No AutoLoad essa etiqueta precisa estar igual nos quatro motores (backend, frontend, EdgeGateway e portal). Hoje ela está bagunçada — o backend praticamente **não marca produção** e o frontend mistura vários endereços no mesmo projeto. É como receber encomendas de quatro lojas sem saber de qual cada uma veio. Esta etapa garante que cada erro venha com o ambiente certo.

Onde ela atua no código?

- **Backend** (.NET 8): lê a etiqueta em `Program.cs:61` (a partir de `Sentry:Environment`).
- **EdgeGateway** (.NET 8 Worker): lê em `Worker/Program.cs:19` (também de `Sentry:Environment`).

- A **especificação oficial** (`AUTOLOAD_V4_Specs/ ... /20260303_sentry_environment_tags/ESPECIFICACAO.md`) define que os valores corretos são **`dev` / `hml` / `prd`** — e pede que a fonte da etiqueta passe a ser `Logging:Automind:Context:env`. O KIOSK é um módulo à parte (React/Vite), separado por outra tag.

O que ela afeta na prática?

Afeta a **confiança em tudo que vem depois**. Se os erros de teste e de produção ficam misturados, ninguém sabe se um alarme é grave (cliente afetado) ou inofensivo (alguém testando). Arrumar isso é o que permite confiar nos números e nos alertas mais tarde.

Status segundo a auditoria de código (08/06/2026)

⚠️ Divergente O código **já lê** a etiqueta `environment` no backend e no Edge — então não é "do zero". Mas há duas divergências: (1) os valores que o cronograma cita (`production, staging, development, kiosk`) **não batem** com os valores canônicos da especificação (`dev, hml, prd`); e (2) o código lê de `Sentry:Environment`, enquanto a spec pede `Logging:Automind:Context:env`.

Observação extra

O cronograma marca esta etapa como **"Não iniciado"**, mas isso é mais pessimista do que a realidade: parte já existe no código. O que **não dá para provar pelo repositório** é a **cobertura real** — a afirmação de que o backend manda 0% dos eventos com a etiqueta de produção é um estado de **runtime**, que só se confirma olhando o painel do Sentry. Resumindo: falta padronizar valor e fonte da etiqueta, e comprovar a cobertura no painel.

Etapa 2 — Padronizar a "versão" (release), apontar o commit culpado e isolar a esteira

O que é essa etapa?

Sempre que o sistema é publicado, ele ganha um número de **versão** (chamado de *release*). Quando um erro acontece, é muito útil saber **em qual versão** ele surgiu — e, melhor ainda, **qual alteração de código** provavelmente o causou (isso se chama *suspect commits*, o "commit suspeito"). Esta etapa padroniza esse número, liga o "commit suspeito" e impede que a **máquina do desenvolvedor** mande dados para o projeto de produção (isolar a "esteira" de publicação — o pipeline ADO).

Onde ela atua no código?

A versão **já existe** no código: - **EdgeGateway**: `Worker/Program.cs:40` define `options.Release = "AutoloadV4.EdgeGateway.Worker@<versão>"` . - **Frontend**: a versão é injetada no build pelo `withSentryConfig` (`next.config.mjs`). - **Backend**: não define a versão na mão; confia na detecção automática de `SENTRY_RELEASE` (formato `<componente>@<hash>` , documentado em `Autoload_V4-documentacao.md:287`).

O que ela afeta na prática?

Afeta a **velocidade para achar a causa de um problema**. Com a versão e o commit suspeito configurados, quando algo quebra o time descobre em minutos qual mudança causou. Sem isso, vira investigação no escuro. Também é o que permite, mais tarde, cruzar **picos de erro com deploys**.

Status segundo a auditoria de código (08/06/2026)

⚠️ Divergente (já parcialmente no código) O cronograma chega a dizer que a release **já existe** (`SENTRY_RELEASE`) — e a auditoria confirma. Então o trabalho real **não é criar**, e sim: **padronizar o formato** em todos os componentes, ativar o `set-commits --auto` (o commit suspeito) e **isolar a esteira** para que máquinas de desenvolvimento não enviem ao projeto produtivo. A famosa "inconsistência Linux (ADO) + Windows (dev) no mesmo projeto" é estado de **pipeline/painel → não verificável no código**.

Observação extra

É um **bloqueador central**: enquanto a release não estiver padronizada, o "commit suspeito", a detecção de regressão e a correlação picos×deploys ficam todos travados. A recomendação da auditoria é tornar a release **explícita também no backend** (hoje ele depende de adivinhação), alinhando com o Edge.

Etapa 3 — Isolar o ambiente de desenvolvimento do de produção

O que é essa etapa?

A máquina do desenvolvedor, rodando o sistema localmente (`localhost`), **não deveria** mandar seus erros para o mesmo lugar onde ficam os erros dos clientes reais. Quando isso acontece, o painel de produção fica poluído com "erros" que são apenas alguém programando. Esta etapa garante que o desenvolvimento e a produção fiquem **separados**.

Onde ela atua no código?

O cronograma descrevia o mecanismo como `enabled = (NODE_ENV === 'production')` num arquivo chamado `sentry.client.config.ts` . A auditoria mostrou que, na prática, é diferente: - O arquivo do cliente é `src/instrumentation-client.ts` (padrão do Next 14). - O "freio" real é a **validação do endereço do Sentry (DSN)**: se o DSN ainda contém o marcador `#` (ou seja, não foi preenchido na publicação), o SDK simplesmente **não liga** (fica "no-op"). - No **backend** e no **Edge**, vale a mesma ideia: quando `Sentry:Dsn` é um placeholder (`#{ ... }#`), o monitoramento fica desativado.

O que ela afeta na prática?

Afeta a **limpeza do painel de produção**. Um `localhost` despejando erros no projeto produtivo mistura barulho de desenvolvimento com problemas reais de cliente.

Status segundo a auditoria de código (08/06/2026)

⚠️ Divergente (mecanismo já existe, descrito errado) O isolamento **já está implementado** no código — só que por um caminho diferente do que o cronograma dizia (gate por **DSN placeholder**, não por `NODE_ENV` ; arquivo `instrumentation-client.ts` , não `sentry.client.config.ts`). A auditoria corrigiu o nome do arquivo e o mecanismo no cronograma.

Observação extra

O cronograma marca "Não iniciado", mas o freio por DSN já existe. O que falta é **garantir a configuração de publicação**: que o ambiente de desenvolvimento receba mesmo um DSN nulo/placeholder, para que o freio sempre atue.

Etapa 4 — Corrigir o "bug" do MQTT (handler de mensagens)

O que é essa etapa?

O EdgeGateway conversa com os equipamentos de campo por um protocolo chamado **MQTT** (como uma estação de rádio em que mensagens são publicadas e ouvidas). Para ouvir, o programa "assina" um canal com um *handler* (um ouvinte). O cronograma afirmava que, na hora de **cancelar** essa assinatura, o código usava `+=` (que **adiciona** outro ouvinte) em vez de `-=` (que **remove**) — o que causaria acúmulo de ouvintes (*memory leak*) e processamento **duplicado** das mensagens.

Onde ela atua no código?

A auditoria abriu os dois serviços e encontrou o **oposto** do que o cronograma dizia: - **Backend**: `AutoloadV4.Domain.Operations/Services/MQTTService.cs:106` remove com `ApplicationMessageReceivedAsync -= _currentHandler;` - **EdgeGateway**: `AutoloadV4.EdgeGateway/Services/MQTTService.cs:98` faz a mesma coisa. - O `+=` que aparece (`:91` / `:85`) é a **assinatura** normal — e, antes dela, o código já faz o `-=` do ouvinte anterior. O comportamento está inclusive documentado nos comentários do código.

O que ela afeta na prática?

Na prática, nada precisa ser corrigido — porque o problema apontado **não existe**. O código já faz o certo. Se existisse, seria um vazamento de memória e mensagens processadas em dobro.

Status segundo a auditoria de código (08/06/2026)

🟦 Improcedente / já concluído Esta é a constatação **mais importante** da auditoria. Este item era citado nos dois cronogramas como a correção de **"custo trivial, impacto direto"** — a "vitória fácil" da semana 1. Aberto o código, a premissa **não se confirma**: o `-=` já está lá. O item foi **reclassificado como encerrado** (criticidade baixa). A única ação residual sugerida é um **teste de regressão** que prove o ciclo assinar/cancelar.

Observação extra

Vale a transparência: este era o exemplo mais repetido de "quick win" da Onda 0, e ele simplesmente **já estava feito**. Confiar no cronograma sem abrir o código teria gerado retrabalho sobre um bug que não existe.

Etapa 5 — Quebrar o loop ruidoso `EDGEGATEWAY-2V`

O que é essa etapa?

Existe **uma única falha** no EdgeGateway (uma `ServiceException` com a mensagem "A transação deve...") que dispara sem parar: cerca de **14.686 eventos em 30 dias**, vindos de **um só usuário**. No ritmo atual, isso projeta **~440 mil eventos por mês** saindo de **um único erro** — o que pode **estourar a cota** contratada do Sentry (e custar dinheiro), além de afogar o painel em barulho. Esta etapa investiga e estanca esse loop.

Onde ela atua no código?

Esta etapa **não tem um ponto fixo no código** para "consertar de imediato": é preciso primeiro **investigar a causa-raiz** da `ServiceException`. Se a raiz não puder ser corrigida rápido, o plano B é aplicar um `beforeSend` com **amostragem de 1 a cada 100** ocorrências daquele erro específico (*fingerprint*), abrindo um ticket de débito técnico.

O que ela afeta na prática?

Afeta **custo e visibilidade**. Um erro que sozinho projeta centenas de milhares de eventos/mês consome cota e esconde os problemas de verdade no meio da enxurrada.

Status segundo a auditoria de código (08/06/2026)

❓ Não verificável no código A contagem de eventos, o usuário afetado e a projeção de cota são **métricas do painel do Sentry**, não coisas que o repositório prove. A auditoria não consegue confirmar nem o volume nem a correção pelo código — segue como **pendência real de painel/investigação**.

Observação extra

O cronograma marca "Não iniciado", e aqui isso provavelmente reflete a realidade: é uma das etapas em que **a ação ainda precisa começar**. Pelo risco de cota, é uma das prioridades concretas da Onda 0.

Etapa 6 — Filtrar o que sai (`beforeSend`) e proteger dados pessoais (LGPD)

O que é essa etapa?

Quando um erro acontece, o Sentry pode capturar dados que estavam em jogo — e no AutoLoad isso inclui **dados sensíveis**: CPF, placa de caminhão, e-mail e informações do motorista. Esta etapa instala um "filtro de saída" (`beforeSend`) que **borra/remove** esses dados **antes** de enviá-los, desliga o envio automático de informação pessoal (`SendDefaultPii`) e configura regras de limpeza também do lado do servidor do Sentry (Data Scrubber). Há contratos sensíveis no meio (Ultracargo, Petrobras), então o risco é **legal**, não só técnico.

Onde ela atua no código?

- **Frontend**: **já tem** um `beforeSend` que limpa chaves sensíveis (`password|token|secret|authorization|api_key|bearer|credential` → `[REDACTED]`) nos três modos (`instrumentation-client.ts` , `sentry.server.config.ts` , `sentry.edge.config.ts`). Porém usa `sendDefaultPii: true` (ligado) — o oposto do que a etapa pede.
- **EdgeGateway**: já tem `SetBeforeSend` com tags (`serviceName/siteId/gatewayId/tenantId`) em `Worker/Program.cs:43-50` e `SendDefaultPii: false` no `appsettings.json` .

- **Backend**: **não tem** `beforeSend` — captura os erros via `ExceptionMiddleware.cs:13-22`.

O que ela afeta na prática?

Afeta **conformidade legal (LGPD/ANP) e privacidade**. Documentos e dados pessoais podem estar sendo gravados num sistema externo. É um **risco jurídico real**, agravado pelos contratos com grandes clientes.

Status segundo a auditoria de código (08/06/2026)

⚠️ Divergente (parcialmente feito) O frontend e o Edge **já têm** filtro de saída — então não é "zero". A **lacuna real** é dupla: (1) o **backend (.NET)** ainda **não tem** `beforeSend`; e (2) o scrubbing existente cobre **chaves de credencial**, mas **não** os **dados de domínio** (CPF, placa, motorista). Além disso, o `sendDefaultPii: true` do frontend precisa de **revisão sob a ótica da LGPD**. As regras de Data Scrubber na interface do Sentry são **painel** → não verificáveis no código.

Observação extra

É a etapa mais sensível da Onda 0, pelo risco legal. Diferente do AutoChecker (onde estava tudo em aberto), aqui há **proteção parcial** já no código — mas o ponto mais perigoso (dados pessoais do negócio e o backend sem filtro) **continua descoberto**.

Etapa 7 — Corrigir os "source maps" do Frontend

O que é essa etapa?

O código do frontend é "embaralhado" (minificado) antes de ir para produção, para ficar leve. Quando um erro acontece, o que aparece no Sentry vira algo ilegível, como `Object.s` — sem dizer **qual arquivo** ou **qual linha**. Os *source maps* são um "dicionário de tradução" que desembaralha isso e mostra o erro no código original. Hoje há **60 issues "Object.s"** e mais uma `"<unknown>"` sem rastro legível. Esta etapa garante que o tradutor seja enviado e usado corretamente.

Onde ela atua no código?

A instrumentação **já existe**: `next.config.mjs:16-52` usa `withSentryConfig( ... )` com `widenClientFileUpload: true` (`:31`) e `silent: !process.env.CI` (`:25`) — ou seja, **faz o upload do tradutor durante a publicação no CI**. O `hideSourceMaps` explícito não está setado (usa o padrão).

O que ela afeta na prática?

Afeta a **legibilidade dos erros do frontend**. Sem o tradutor, cada erro de tela vira um enigma e o time perde tempo só para descobrir **onde** o problema aconteceu.

Status segundo a auditoria de código (08/06/2026)

⚠️ Divergente (já instrumentado no código) O upload dos source maps **já está configurado** no `next.config.mjs`. Portanto o sintoma `Object.s` provavelmente **não é falta de configuração no código**, e sim **consistência de `release` /token de autenticação na esteira** (o tradutor enviado precisa casar com a versão que roda em produção) — e isso é **pipeline/painel** → não verificável no código.

Observação extra

O cronograma marca "Não iniciado", mas no código o upload está lá. A pendência verdadeira mora na **esteira de publicação** (alinhar release e token), não no repositório do frontend.

Etapa 8 — Fazer a faxina (triagem) do backlog de erros (349 issues)

O que é essa etapa?

"Triagem" é como a fila de emergência de um hospital: olhar cada caso e classificar — **resolvido**, **ignorar** (com prazo) ou **em andamento** — e anotar o critério usado. Hoje há cerca de **349 erros em aberto** no Sentry e **zero foram resolvidos nos últimos 30 dias**. É uma caixa de entrada que ninguém está esvaziando. Esta etapa coloca a casa em ordem.

Onde ela atua no código?

Esta etapa **não mexe no código** — é trabalho de organização feito **dentro do painel do Sentry** (classificar issues, definir critérios, atribuir donos). Ela se beneficia das outras etapas: resolver as causas (loops, PII, source maps) é o que faz a fila diminuir de verdade.

O que ela afeta na prática?

Afeta a **rotina de cuidado com o sistema**. Um backlog sem dono **contamina qualquer métrica** que se queira tirar depois (não dá para medir "quantos erros novos por semana" se a base nunca é limpa).

Status segundo a auditoria de código (08/06/2026)

❓ Não verificável no código (processo/painel) A contagem de 349 issues e o "0 resolvidas em 30 dias" são estado do **painel do Sentry** — não há nada no repositório que prove ou conteste. É uma pendência de **processo**, que precisa de pessoas e critério, não de código.

Observação extra

Como no AutoChecker, a triagem **ainda não começou de verdade**. É a etapa mais "humana" da Onda 0 e, sozinha, não corrige nada — mas sem ela nenhuma métrica derivada terá valor.

Resumo geral da Onda 0

#	Nome da etapa	Status segundo a auditoria de código	Risco principal
1	Padronizar etiqueta `environment` (4 SDKs)	⚠️ Divergente (lê no código; valores `dev/hml/prd` e fonte divergem)	Backend sem cobrir produção; cobertura só se prova no painel
2	Release + commit suspeito + isolar esteira	⚠️ Divergente (release já existe; falta padronizar/ `set-commits`)	Bloqueia commit suspeito, regressão e correlação picos×deploys

#	Nome da etapa	Status segundo a auditoria de código	Risco principal
3	Isolar desenvolvimento de produção	⚠️ Divergente (gate por DSN já existe; descrito errado)	`localhost` pode poluir o projeto produtivo
4	Corrigir "bug" do MQTT (`+=` → `-=`)	🟦 Improcedente / já concluído (código já usa `-=`)	Nenhum — o bug apontado não existe; falta só teste de regressão
5	Quebrar o loop `EDGEGATEWAY-2V`	❓ Não verificável no código (métrica do painel)	~440k eventos/mês de UMA issue → risco de estouro de cota
6	`beforeSend` + proteção de PII (LGPD)	⚠️ Divergente (FE/Edge ok; **backend e PII de domínio em aberto**)	**Risco legal:** CPF/placa/motorista; contratos sensíveis
7	Corrigir source maps do Frontend	⚠️ Divergente (upload já instrumentado no código)	60 issues `Object.s` ilegíveis; causa está na esteira
8	Triagem do backlog (349 issues)	❓ Não verificável no código (processo/painel)	Backlog sem dono contamina qualquer métrica; 0 resolvidas

Contagem por situação (segundo a auditoria de código, 08/06/2026): - 🟦 Improcedente / já concluído: **1** (Etapa 4) - ⚠️ Divergente — já parcialmente no código, com pendência clara: **5** (Etapas 1, 2, 3, 6, 7) - ❓ Não verificável no código (depende de painel/runtime/processo): **2** (Etapas 5 e 8) - ✅ Totalmente concluída e comprovável só pelo código: **0**

Conclusão

> **A Onda 0 NÃO pode ser considerada concluída, e o gate deve permanecer FECHADO — mas o quadro é mais favorável (e mais específico) do que o cronograma sugere.**

A leitura honesta da auditoria é dupla. De um lado, o cronograma marca **todas as 8 etapas como "Não iniciado"**, o que é **pessimista demais**: ao abrir o código, várias peças **já existem** — a leitura do `environment` , a `release` , o isolamento por DSN, o `beforeSend` do frontend e do

Edge, e o upload de source maps. E um item inteiro (o "bug" do MQTT) era **improcedente**: já estava correto.

De outro lado, o gate **não fecha** porque o que falta é justamente o que **não se prova só pelo código** e os pontos realmente sensíveis: a **cobertura real** da etiqueta de produção (Etapa 1), o **loop de cota** `EDGEGATEWAY-2V` (Etapa 5), o **buraco de LGPD** no backend e nos dados de domínio (Etapa 6), a **consistência da esteira** para os source maps (Etapa 7) e a **triagem** que nunca começou (Etapa 8).

> **Recomendação:** em vez de "começar as 8 do zero", o time deveria (1) tratar com **urgência** a Etapa 6 (dados pessoais — risco legal), (2) atacar o **loop de cota** da Etapa 5 antes que vire custo, (3) **padronizar release + set-commits** da Etapa 2 (que destrava várias outras), e (4) **alinhar valores e fonte** da etiqueta `environment` (Etapa 1), comprovando a cobertura no painel. As Etapas 3, 4 e 7 estão muito perto de "pronto" e pedem apenas confirmação/teste.

Balanço final

- **Etapas já parcialmente resolvidas no código (precisam de ajuste, não de criação): 5** (Etapas 1, 2, 3, 6 e 7).
- **Etapa improcedente (apontava um bug que não existe): 1** (Etapa 4).
- **Etapas que dependem de painel/processo (fora do alcance do código): 2** (Etapas 5 e 8).
- **Etapas totalmente concluídas e comprováveis pelo código: 0**.
- **O gate da Onda 0 pode ser liberado? NÃO.** O portão permanece fechado — mas a lista real de pendências é menor e mais precisa do que "8 etapas não iniciadas".

Documento explicativo gerado em 17/06/2026. Fontes: Cronograma Revisado (`Cronograma_Observabilidade_AutoLoad_V4_Revisado_2026-06-02.xlsx` *), Auditoria de código (* `Auditoria_Codigo_Cronograma_Observabilidade_AutoLoad_V4_2026-06-08.md` *), documentação do produto, ata de alinhamento e questionário do desenvolvedor. Como o AutoLoad não possui board no Azure DevOps, todos os status vêm da auditoria que confrontou o cronograma com o código real; onde a verdade depende do painel do Sentry ou do runtime, isso está registrado explicitamente como "não verificável no código".*